$SiER^2NA$

Simple Extensible Register-Register Neumann Architecture

Simple Parallel Peripheral Bus Reference

1.0

October 25, 2026

Contents

1 Preface **2**

2 Glossary **2**

3 Introduction **2**

4 Host Controller **2**
- 4.1 Peripheral Memory Map Table . . . 3
- 4.2 Peripheral Memory Map Entry . . . 3

5 Bridge **3**

6 Device **4**

7 Protocol **4**
- 7.1 Request . . . 5
  - 7.1.1 0x0 Read Byte Request . . . 5
  - 7.1.2 0x1 Read Half-Word Request . . . 5
  - 7.1.3 0x2 Read Word Request . . . 5
  - 7.1.4 0x3 Write Byte . . . 5
  - 7.1.5 0x4 Write Half-Word . . . 5
  - 7.1.6 0x5 Write Word . . . 5
  - 7.1.7 0x6 Interrupt Information Request . . . 6
- 7.2 Response . . . 6
  - 7.2.1 0x0 Read Byte Response . . . 6
  - 7.2.2 0x1 Read Half-Word Response . . . 6
  - 7.2.3 0x2 Read Word Response . . . 6
  - 7.2.4 0x6 Interrupt Information Response . . . 6

8 Interrupts **6**

1 Preface

2 Glossary

Glossary

Bus ID Index of a physical bridge port that a device can connect to. 3, 4

Port ID Index of a virtual I/O port in a device, corresponding to a specific address region. 3, 4

Acronyms

SPPB Simple Parallel Peripheral Bus. 2–4

3 Introduction

The Simple Parallel Peripheral Bus (SPPB) is a parallel data bus and control interface designed for connecting external devices with the CPU. It is design to be able to address a maximum of 16 connected devices at once with each connected device being identified by a 4-bit ID indicating the bus slot it is connected to.

It utilises point-to-point connections for the physical communication layer.

4 Host Controller

The SPPB controller is a component of the CPU that translates memory I/O requests from the MMU to electrical signals for the SPPB bridge.

It internally uses a memory map that maps memory addresses to respective device ports, which is hardware-mapped as a static physical address and is accessible using standard load/store instructions.

The resulting signals from the controller are outputted via GPIO on the CPU and received by the bus bridge.

Table 1: Physical interface

GPIO pin	Description
0	Data 0
1	Data 1
2	Data 2
3	Data 3
4	Data 4
5	Data 5
6	Data 6
7	Data 7
8	Clock
9	Interrupt request

4.1 Peripheral Memory Map Table

Offset	Value
0x00	Entry 0
0x08	Entry 1
0x10	Entry 2
...	...
0x1F8	Entry 63
0x200	

4.2 Peripheral Memory Map Entry

63..44	43..24	23..20	19..16	15..3	2..1	0
&bs	&lm	bid	pid	–	rw	p
Physical base address[31:12]	Physical limit address[31:12]	Bus ID	Port ID	Reserved	Read/Write	Present

NOTE: Both the base and limit addresses must be `0x1000` aligned.

NOTE: Base address is inclusive, while limit is exclusive, i.e. $base \leq addr < limit$

NOTE: *Read/write* select what I/O operation the port is used for. It can be `1` for read, `2` for write, or `4` for both read and write.

5 Bridge

The SPPB bridge is, in principle, a multiplexer with a digital control protocol. It's responsible for routing memory I/O between the system board and connected peripherals.

The physical interface that connects the bridge to the system board is a 2x12-pin connector.

Table 2: Physical bridge interface

Description	Pin		Pin		Description
Ground	GND	0	1	D0	Data 0
Data 1	D1	2	3	D2	Data 2
Data 3	D3	4	5	GND	Ground
Ground	GND	6	7	D4	Data 4
Data 5	D5	8	9	D6	Data 6
Data 7	D7	10	11	GND	Ground
Ground	GND	12	13	CLK	Clock
INT select 0	IS0	14	15	IS1	INT select 1
INT select 2	IS2	16	17	IS3	INT select 3
INT select 4	IS4	18	19	GND	Ground
INT select 5	IS5	20	21	IS6	INT select 6
INT request	IRQ	22	23	IS7	INT select 7

Upon receiving a request packet from a *host controller*, the bridge decodes the *request frame* (See 7), reads the Bus ID and Port ID and, using its multiplexer or similar, routes the packet to the specified device.

Afterwards, if the request packet requires a response, it waits for the device to communicate it and is then sent directly back to the *host controller*.

6 Device

SPPB devices respond to requests from the *bridge* and do not send any data on their own. They are connected to the *bridge* with a 2x10-pin connector.

A compliant device must also return to a clean state when receiving a HIGH signal on the `RST` pin from the bridge.

Table 3: Physical device interface

Description	Pin		Pin		Description
Ground	GND	0	1	D0	Data 0
Data 1	D1	2	3	D2	Data 2
Data 3	D3	4	5	GND	Ground
Ground	GND	6	7	D4	Data 4
Data 5	D5	8	9	D6	Data 6
Data 7	D7	10	11	GND	Ground
–	–	12	13	–	–
Ground	GND	14	15	CLK	Clock
Reset	RST	16	17	–	–
INT request	IRQ	18	19	GND	Ground

7 Protocol

SPPB uses a stateless, request-response protocol. The bridge and connected devices do not send data on their own and only respond to requests by the bridge or host controller.

Every request packet is preceded by a request frame, which is a 16-bit struct that specifies basic information about the request:

Table 4: Request frame structure

15..12	11..8	7..4	3..0
–	pkt	pid	bid
Reserved	Packet ID	Port ID	Bus ID

As response packets get sent by the device and propagated to the bridge/controller only after receiving a request packet, they contain only data.

NOTE: All addresses encoded into packets are relative from the mapped address base. For example, a read request to address `0xA001 44FF` for a device port mapped at `0xA000 0000-0xA100 0000` would be encoded and sent as `0x0001 44FF`.

7.1 Request

7.1.1 0x0 Read Byte Request

47..16	15..0
adr	rf
Target address	Request frame (7)

7.1.2 0x1 Read Half-Word Request

47..16	15..0
adr	rf
Target address	Request frame (7)

7.1.3 0x2 Read Word Request

47..16	15..0
adr	rf
Target address	Request frame (7)

7.1.4 0x3 Write Byte

This request does not have a response packet

55..48	47..16	15..0
d	adr	rf
Data	Target address	Request frame (7)

7.1.5 0x4 Write Half-Word

This request does not have a response packet

63..48	47..16	15..0
d	adr	rf
Data	Target address	Request frame (7)

7.1.6 0x5 Write Word

This request does not have a response packet

79..48	47..16	15..0
d	adr	rf
Data	Target address	Request frame (7)

7.1.7 0x6 Interrupt Information Request

15..0
rf
Request frame (7)

7.2 Response

7.2.1 0x0 Read Byte Response

7..0
d
Data

7.2.2 0x1 Read Half-Word Response

15..0
d
Data

7.2.3 0x2 Read Word Response

31..0
d
Data

7.2.4 0x6 Interrupt Information Response

23..16	15..0
int	rf
Interrupt ID	Request frame (7)

If the device did not raise any interrupt, `int` is 0.

8 Interrupts

Interrupts are first handled within the device-bridge loop before being sent to the CPU as a binary selector.

Whenever a connected device wants to raise an interrupt, it communicates the need to do so by pulling the `IRQ` pin HIGH on the bridge. When the bridge detects the `IRQ` pin set HIGH, it finishes the current operation (if any) and sents out a Interrupt Information Request packet to every connected device in an unspecified order, propagating the interrupt request to the CPU after decoding the response packet and if `int` is not zero. The `IRQ` pin is then cleared by the requesting device after receiving the IIRQ packet.

If `IRQ` is still HIGH after receiving a non-zero interrupt ID, i.e. more than one device wants to raise an interrupt at the same time, the *bridge* completes a single queued request (if any) from the *host controller* before going through the same process.